

1. Produtor-Consumidor e Fila

Arquivo: 04_queue_producer_consumer.py

```
C:\Python\rt_python_demos>python 04_queue_producer_consumer.py
[PRODUTOR] Produziu: 1 | Fila: [1]
[CONSUMIDOR] Consumiu: 1 | Fila: []
[PRODUTOR] Produziu: 2 | Fila: [2]
[PRODUTOR] Produziu: 3 | Fila: [2, 3]
[CONSUMIDOR] Consumiu: 2 | Fila: [3]
[PRODUTOR] Produziu: 4 | Fila: [3, 4]
[PRODUTOR] Produziu: 5 | Fila: [3, 4, 5]
[CONSUMIDOR] Consumiu: 3 | Fila: [4, 5]
[PRODUTOR] Produziu: 6 | Fila: [4, 5, 6]
[CONSUMIDOR] Consumiu: 4 | Fila: [5, 6]
[PRODUTOR] Produziu: 7 | Fila: [5, 6, 7]
[PRODUTOR] Produziu: 8 | Fila: [5, 6, 7, 8]
[CONSUMIDOR] Consumiu: 5 | Fila: [6, 7, 8]
[PRODUTOR] Produziu: 9 | Fila: [6, 7, 8, 9]
[CONSUMIDOR] Consumiu: 6 | Fila: [7, 8, 9]
....
... Execução finalizada.
```

2. Bloqueio por Mutex

Arquivo: 05_mutex_blocking_latency.py

```
C:\Python\rt_python_demos>python 05_mutex_blocking_latency.py

--- Teste de Mutex e Latência ---

[T1] Tentando acessar o recurso...
[T1] Acessou o recurso após 0.000 s
[T1] Liberou o recurso.
[T2] Tentando acessar o recurso...
[T2] Acessou o recurso após 1.501 s
[T2] Liberou o recurso.
[T3] Tentando acessar o recurso...
[T3] Acessou o recurso após 3.001 s
[T3] Liberou o recurso.

Latências observadas: [0.0, 1.501, 3.001] segundos
Latência média: 1.500 s
```

3. Inversão de Prioridade

Arquivo: 06_priority_inversion_simulated.py

```
C:\Python\rt_python_demos>python 06_priority_inversion_simulated.py

--- Simulação de Inversão de Prioridade ---

[L] (baixa) iniciou e está usando o recurso.
[H] (alta) tentando acessar o recurso... BLOQUEADA!
[M] (média) executando...
[M] (média) executando...
[M] (média) executando...
[L] (baixa) liberou o recurso.
[H] (alta) conseguiu acessar o recurso após espera!

Inversão de prioridade observada com sucesso.
```

4. Escalonamento Rate Monotonic (RM)

Arquivo: 07_rate_monotonic_sim.py

```
C:\Python\rt_python_demos>python 07_rate_monotonic_sim.py

--- Escalonamento Rate Monotonic ---

Tempo  0  1  2  3  4  5  6  7  8  9  9  10 11 12 13 14 15 16 17 18 19 20
Tarefa A (T=4) : A A A A A A A A
Tarefa B (T=5) : B B B B B B B B
Tarefa C (T=10): C C C C C C C C
CPU           : A A A B A A C B A B A C A B A A A C A B

Total de Deadlines perdidos: 0
Escalonamento RM finalizado.
```

7. Malha de Controle Periódica

Arquivo: 10_control_loop_fixed_period.py

```
C:\Python\rt_python_demos>python 10_control_loop_fixed_period.py

--- Malha de Controle com Período Fixo ---

Iteração | Tempo (s) | Erro | Saída
-----|-----|-----|-----
0         | 0.000     | 1.000 | 0.500
1         | 0.500     | 0.600 | 0.800
2         | 1.000     | 0.360 | 0.920
3         | 1.500     | 0.216 | 0.976
4         | 2.000     | 0.130 | 0.988
5         | 2.500     | 0.078 | 0.994
6         | 3.000     | 0.047 | 0.997
7         | 3.500     | 0.028 | 0.999
8         | 4.000     | 0.017 | 0.999
9         | 4.500     | 0.010 | 1.000

Malha finalizada.
```

5. Escalonamento EDF

Arquivo: 08_edf_sim.py

```
C:\Python\rt_python_demos>python 08_edf_sim.py

--- Escalonamento EDF ---

Tempo  0  1  2  3  4  5  6  7  8  9  10 11 12 13 14 15 16 17 18 19 20
Tarefa A (D=4) : A A A A A A A A
Tarefa B (D=7) : B B B B B B B B
Tarefa C (D=10): C C C C C C C C
CPU           : A A B A C B B A A C B A B B B A C B

Total de Deadlines perdidos: 0
Escalonamento EDF finalizado.
```

6. Watchdog

Arquivo: 09_watchdog_simulated.py

```
C:\Python\rt_python_demos>python 09_watchdog_simulated.py

--- Simulação de Watchdog ---

[SISTEMA] Funcionando normalmente... (0)
[SISTEMA] Funcionando normalmente... (1)
[SISTEMA] Funcionando normalmente... (2)
[SISTEMA] TRAVOU!
[WATCHDOG] Falha detectada! Reiniciando sistema...
[SISTEMA] Sistema reiniciado com sucesso.
[SISTEMA] Funcionando normalmente... (0)
[SISTEMA] Funcionando normalmente... (1)
[SISTEMA] Funcionando normalmente... (2)

Simulação finalizada.
```

8. Primeiro Contato com Gantt (E25)

Simulação de Gantt simples

```
Diagrama de Gantt (exemplo)

Tempo -> 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
Tarefa A : [AAAA] [ ] [AAAA] [ ]
Tarefa B : [ ] [BB] [ ] [ ] [BB] [ ]
Tarefa C : [ ] [ ] [CCC] [ ] [CCC] [ ]
CPU       : A A A A B B B C C A A A B B C C C

Legenda:
A = Tarefa A (T= 5, C=2)
B = Tarefa B (T= 7, C=2)
C = Tarefa C (T=10, C=3)

Deadlines perdidos: 0
```

9. Offsets e Fase Inicial (E28)

Efeito de offsets nas liberações

```
C:\Python\rt_python_demos>python offsets_sia.py
```

Liberações de tarefas com offsets

Tempo -> 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

Tarefa A (off=0): *

Tarefa B (off=2): *

Tarefa C (off=4): *

Legenda: * = liberação da tarefa

Observação:

Os offsets distribuíram as liberações ao longo do tempo.

10. Conjuntos Harmônicos vs Não Harmônicos (E17)

Comparação de conjuntos de períodos

```
C:\Python\rt_python_demos>python harmonic_vs_nonharmonic.py
```

=== CONJUNTO HARMÔNICO ===

Períodos: [4, 8, 16]

Deadlines perdidos: 0

Utilização: 0.75

=== CONJUNTO NÃO HARMÔNICO ===

Períodos: [5, 7, 11]

Deadlines perdidos: 1

Utilização: 0.78

Conclusão: Conjunto harmônico apresentou comportamento mais previsível e sem perdas de deadlines.